

UNIVERSITAT POLITÈCNICA DE CATALUNYA

School of Professional & Executive Development

Postgrado en

Mobile Apps Design

VIEWS AND LAYOUTS

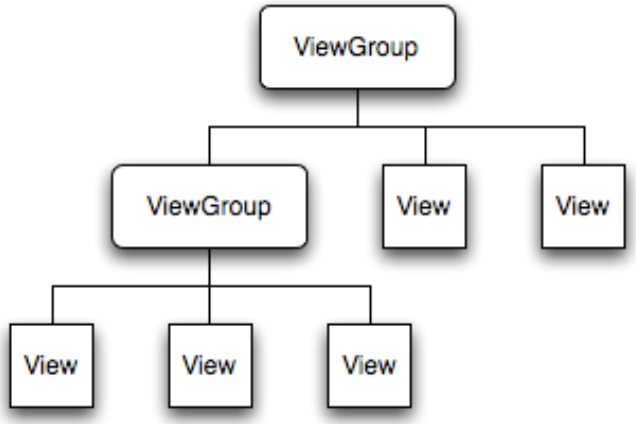
Juan Luis Gorricho

USER INTERFACE

- The user interface is built using **View** and **ViewGroup** objects
- Subclasses of View, the "**widgets**", are implementations of the View class, e.g. Button, TextView, etc.
- Subclasses of ViewGroup, the "**layouts**", implement the arrangement of View elements inside a container (the view group): linear, tabular, relative, etc.

USER INTERFACE

- The UI associated to an activity is defined by a hierarchy of View objects, using a View object as the **root** and parameter of: setContentView(view), this method is usually called from onCreate() to create the activity.



USER INTERFACE

- The **drawing** of the view is made from the root view downwards to the leaves, this way descendent views take their references on **dimensions** and **location** from their parent already drawn.
- The geometry of any view is **rectangular**. The view location is defined by its coordinates: (**left, top**) and dimensions (**width, height**)
- If there's some overlapping, the last drawn views will be shown on top of the previously drawn.

LAYOUTS

- To define a layout there are two methodologies:
  - Using an XML file.
  - By code, instantiating and composing the layout on run time.

DECLARATIVE LAYOUT

- Example of a declarative layout:

```
<?xml version="1.0" encoding="utf-8"?>

    <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
                  android:layout_width="fill_parent"
                  android:layout_height="fill_parent"
                  android:orientation="vertical" >

       <TextView android:id="@+id/text"
                  android:layout_width="wrap_content"
                  android:layout_height="wrap_content"
                  android:text="Hello, I am a TextView" />

       <Button android:id="@+id/button"
              android:layout_width="wrap_content"
              android:layout_height="wrap_content"
              android:text="Hello, I am a Button" />

    </LinearLayout>
```

CODED LAYOUT

```
LinearLayout layout = new LinearLayout(this);
layout.setLayoutParams(new
LinearLayout.LayoutParams(LayoutParams.FILL_PARENT,
LayoutParams.FILL_PARENT));
layout.setOrientation(LinearLayout.VERTICAL);

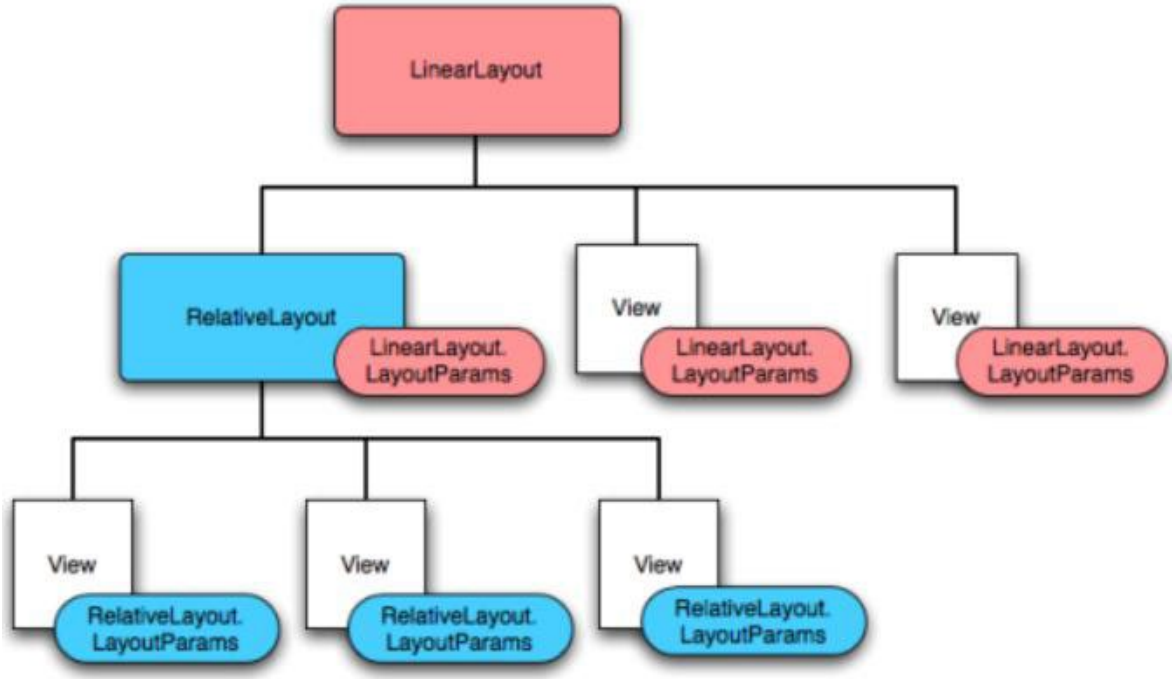
TextView tv = new TextView(this);
tv.setText("Hello, I am a TextView");
tv.setLayoutParams(new
LinearLayout.LayoutParams(LayoutParams.WRAP_CONTENT,
LayoutParams.WRAP_CONTENT));

Button btn=new Button(this);
btn.setText("Hello, I am a Button");
btn.setLayoutParams(new
LinearLayout.LayoutParams(LayoutParams.WRAP_CONTENT,
LayoutParams.WRAP_CONTENT));

layout.addView(tv);
layout.addView(btn);
```

LAYOUT PARAMETERS

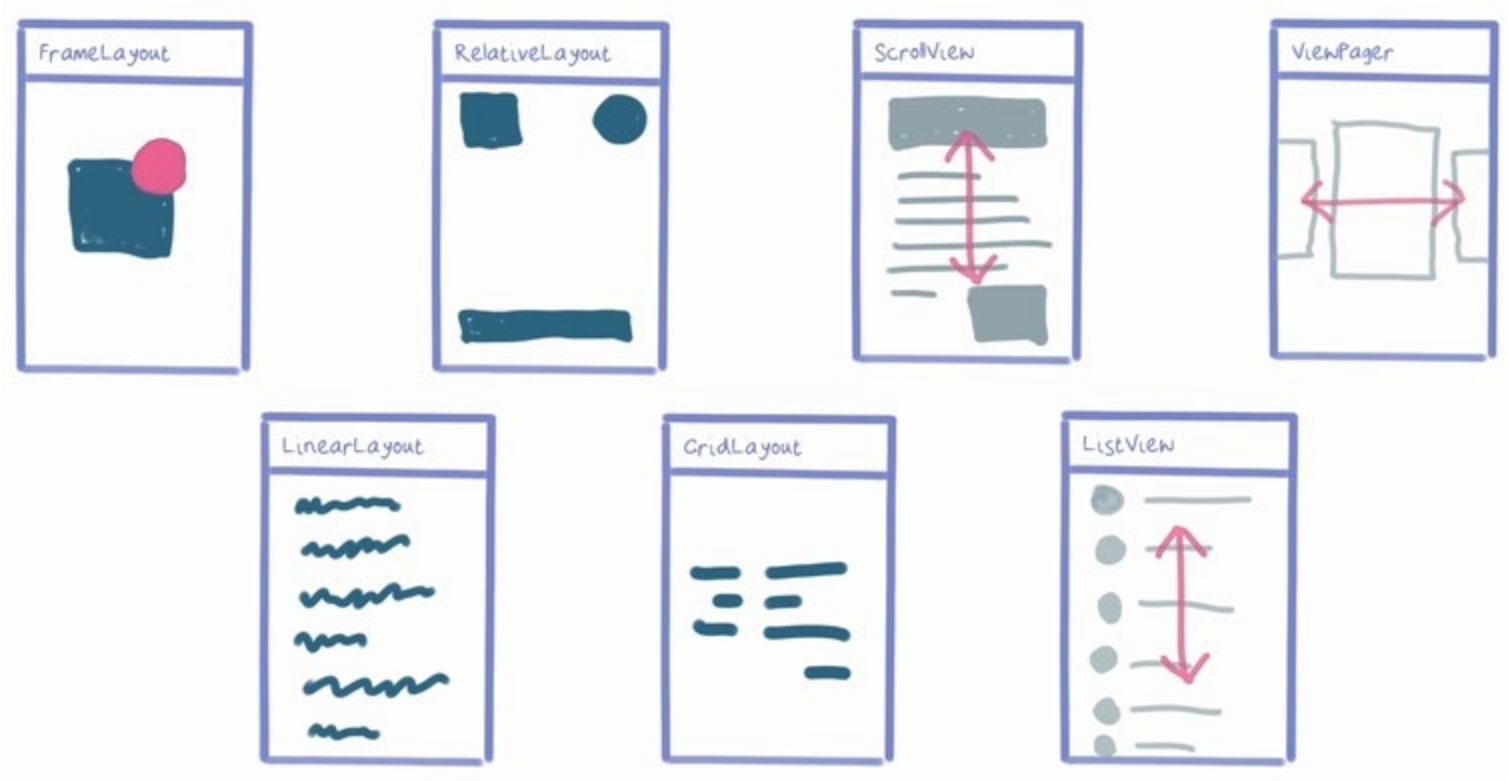
- Any View object can set its **layout parameters** to indicate how to allocate itself inside its container (ViewGroup):



BASIC WIDGETS

- The widgets are a **library** of fully implemented View classes contained within the package: **android.widget**. Examples of basic widgets are:
  - Button, ToggleButton, ImageButton
  - ImageView, TextView, EditText
  - CheckBox, RadioButton
  - CalendarView, DatePicker, etc.

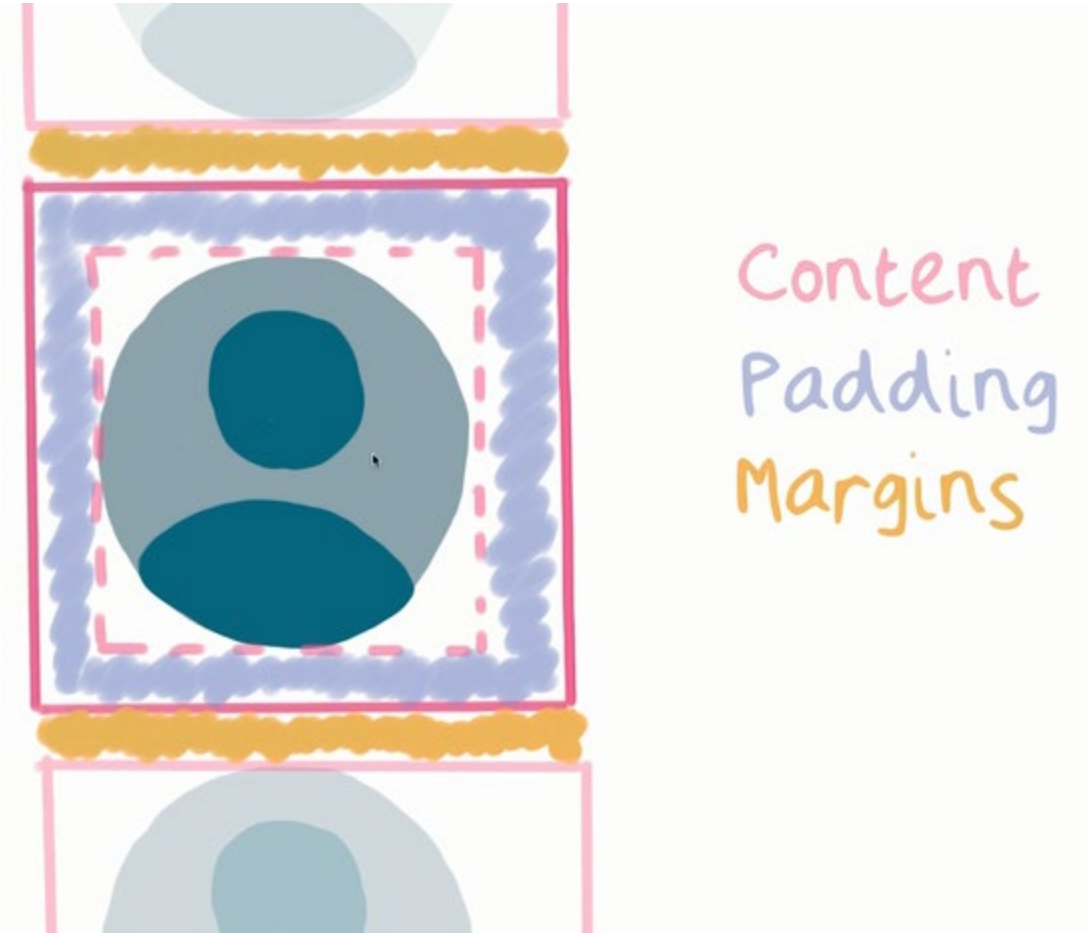
LAYOUTS


FrameLayout
RelativeLayout
ScrollView
ViewPager
LinearLayout
GridLayout
ListView

XML ATTRIBUTES

- **of ViewGroup.LayoutParams:**
  - android:layout_height
  - android:layout_width
- **of ViewGroup.MarginLayoutParams:**
  - android:layout_marginBottom
  - android:layout_marginLeft
  - android:layout_marginRight
  - android:layout_marginTop

PADDING AND MARGINS

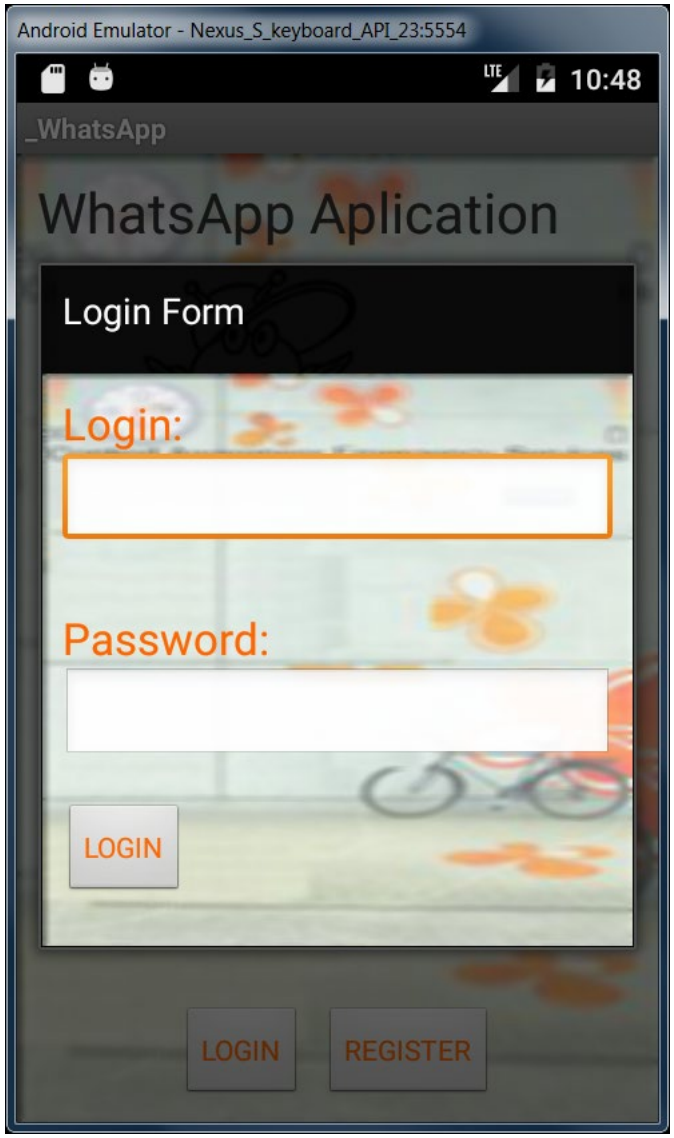


XML ATTRIBUTES

- **of FrameLayout.LayoutParams**:
  - android:layout_gravity
- **of LinearLayout.LayoutParams**:
  - android:layout_gravity
  - android:layout_weight
- **of RelativeLayout.LayoutParams**:
  - android:layout_above
  - android:layout_below
  - android:layout_alignBottom
  - android:layout_alignParentBottom
  - android:layout_toLeftOf
  - etc.

EXERCISE

- Complete the b_login layout to have a form like this:



EXERCISE

- Complete the c_registration layout to have a form like this:

